

PROYECTO INTEGRADOR | BASES DE DATOS AVANZADAS

Data Warehouse del ecosistema futbolistico Transfermarkt

PostgreSQL | Arquitectura Medallion (Bronce, Plata, Oro)
Modelo dimensional en estrella | ETL en Python | Power BI

Documento de sustentacion y guia por diapositiva

Ingenieria de Sistemas | 5to semestre | Universidad Popular del Cesar

Integrantes: [nombres del grupo]

Docente: [nombre del docente]

Valledupar, octubre de 2026

Contenido

- 1. Resumen ejecutivo y contexto 3
- 2. Objetivos y requerimientos 4
- 3. Volumetría y dimensionamiento físico 6
- 4. La base de datos de origen 7
- 5. Diseño del modelo estrella 9
- 6. Arquitectura Medallón 12
- 7. Reglas del proceso EDA 14
- 8. Proceso ETL 15
- 9. Capa Oro: agregaciones, índices y particiones 17
- 10. Dashboard Power BI y storytelling 18
- 11. Guía por diapositiva 20
- 12. Banco de preguntas para la sustentación 23
- 13. Puntos de atención y lista de verificación 25

CAPITULO 1

Resumen ejecutivo y contexto

Que problema se resuelve, que se construye y cuando se entrega

El fútbol profesional toma hoy decisiones de reclutamiento (scouting), valoración de mercado y táctica a partir de grandes volúmenes de datos. Transfermarkt es una de las fuentes públicas más completas: jugadores, clubes, partidos, apariciones, eventos y valoraciones históricas. Sin embargo, analizar ese volumen sobre un esquema transaccional (OLTP) normalizado produce consultas lentas, múltiples JOINS costosos y competencia por recursos con las operaciones de escritura.

Solución propuesta. Construir un Data Warehouse (sistema OLAP) en PostgreSQL con Arquitectura Medallón: una capa Bronce (datos crudos), una Plata (datos validados y tipados) y una Oro (modelo dimensional en estrella y agregaciones precalculadas). Un pipeline ETL en Python mueve y depura los datos entre capas, y Power BI consume la capa Oro para generar un tablero interactivo orientado a decisiones.

1.1 Alcance en una frase

Convertir 10.565.088 registros de Transfermarkt en un modelo en estrella, auditado y rápido (menos de 3 s por consulta), que responda 11 requerimientos funcionales de analítica deportiva desde un dashboard en Power BI.

1.2 Cronograma de entregas del curso

Fecha	Tipo	Contenido exigido	Capítulo de este documento
28 sep	Presentación (5 min)	Introducción, objetivo general, objetivos específicos, requerimientos, volumetría	1, 2, 3 y 11
30 sep	Sustentación	Diseño del modelo estrella del DWH	5 y 11
7 oct	Presentación	Arquitectura Medallón, roles, actividades por capa, herramientas, reglas EDA	6, 7 y 11
12 oct	Sustentación	Proceso ETL: herramientas, flujo de extracción, transformación y carga	8 y 9
14 oct	Sustentación	Dashboards interactivos, filtros claros, diseño para decisiones, storytelling	10
Final	Entrega	Documento PDF con todo el producto	Este documento

CAPITULO 2

Objetivos y requerimientos

Qué se debe lograr y cómo se demuestra

2.1 Objetivo general

Diseñar e implementar una arquitectura de datos escalable basada en la metodología Medallón y un modelo dimensional en estrella, consolidando más de 10,5 millones de registros del ecosistema Transfermarkt en PostgreSQL, para soportar analítica interactiva y toma de decisiones mediante Power BI.

2.2 Objetivos específicos y evidencia de cumplimiento

#	Objetivo	Cómo se cumple	Evidencia para la sustentación
1	Ingesta e integración de datos	Carga de los 7 CSV a Bronze, validación de esquema y expansión controlada a más de 10,5 M de filas en PostgreSQL.	Conteos por tabla (10.565.088), consulta de huérfanos = 0, tabla de auditoría
2	Modelo dimensional	Esquema en estrella con Fact_Events y 7 dimensiones, con llaves sustitutas y FKs.	Diagrama del modelo, scripts DDL, consultas JOIN de ejemplo
3	Pipeline ETL	Scripts Python por chunks (10.000 a 200.000 filas), idempotentes, Bronze a Plata a Oro.	Código del pipeline, log de ejecución, reejecución sin duplicados
4	Visualización	Tablero Power BI conectado a Oro con drill-down/up y segmentadores sincronizados.	Demostración en vivo del dashboard y tiempos de respuesta

2.3 Requerimientos funcionales y cómo se resuelve cada uno

ID	Requerimiento	Dónde se resuelve
RF01	Ranking Top N de jugadores (goles, asistencias, minutos) con filtros de liga y edad	gold.agg_player_season + segmentadores en Power BI
RF02	Métricas técnicas por posición (pases, faltas, tiros, recuperaciones)	Fact_Appearances + dim_player.position (ver alerta de datos, cap. 13)
RF03	Drill-down/up geográfico (Confederación > Liga > Club > Jugador) y temporal (Año > Mes > Partido > Minuto)	Jerarquías en dim_competition, dim_club, dim_player, dim_date y minuto del hecho
RF04	Comparativa local vs visitante	Campo is_home en Fact_Events y gold.agg_home_away
RF05	Correlación entre valor de plantilla y puntos	Fact_Valuations + gold.agg_club_season, función corr()
RF06	Disciplina: tarjetas por 90 min y por árbitro	gold.agg_discipline_ref + dim_referee
RF07	Eventos en intervalos de 15 minutos	minute_bucket en Fact_Events y gold.agg_events_15min
RF08	Scouting sub-23	Edad calculada en gold.agg_player_season, filtro edad menor a 23
RF09	Dashboard Power BI con selectores sincronizados	Capítulo 10: segmentadores globales y sincronización de segmentadores
RF10	Agregaciones materializadas en Oro	Vistas materializadas gold.agg_* (cap. 9)

ID	Requerimiento	Dónde se resuelve
RF11	Auditoría y linaje del ETL	Tabla meta.etl_audit: filas leídas, transformadas, descartadas e insertadas

2.4 Requerimientos no funcionales y cómo se garantizan

ID	Requerimiento	Mecanismo
RNF01	Procesamiento por chunks de 10.000 a 200.000 filas	pandas read_csv(chunksize=...) y carga con COPY
RNF02	Integridad referencial estricta	FOREIGN KEY en Plata y Oro; prueba de huérfanos = 0
RNF03	Más de 10,5 M de registros sin degradación	Índices, particionamiento y pruebas de tiempo
RNF04	Visualización en menos de 3 s	Power BI en modo Import sobre Oro agregada e indexada
RNF05	Fechas ISO-8601 y tipado fuerte	Conversión en Plata: DATE, INTEGER, NUMERIC
RNF06	Tratamiento canónico de nulos	Tabla de reglas de imputación antes de Plata (cap. 7)
RNF07	Idempotencia	TRUNCATE en Bronce, UPSERT (ON CONFLICT) en Plata y Oro
RNF08	Almacenamiento físico optimizado	Índices B-Tree/Hash y partición por rango en Fact_Events
RNF09	Ingesta tolerante a fallos	try/except por codificación, delimitador y filas corruptas
RNF10	Conector nativo PostgreSQL a Power BI	Conector PostgreSQL de Power BI Desktop

CAPITULO 3

Volumetría y dimensionamiento físico

Auditoría real sobre pg_class y pg_total_relation_size()

La volumetría se midió directamente en el catálogo de PostgreSQL con **pg_class**, **pg_relation_size()** y **pg_total_relation_size()**. Esto evidencia que las cifras no son estimaciones sino mediciones físicas.

Tabla	Registros	Datos (MB)	Índices (MB)	Total (MB)	% de filas
competitions	43	0,016	0,032	0,048	0,0004 %
clubs	426	0,080	0,064	0,144	0,004 %
players	30.499	4,80	2,10	6,90	0,29 %
games	67.421	18,50	8,20	26,70	0,64 %
player_valuations	1.385.000	85,00	42,00	127,00	13,11 %
appearances	1.894.350	285,00	142,00	427,00	17,93 %
game_events	7.187.349	1.220,60	591,60	1.812,20	68,03 %
Total	10.565.088	1.614,00	786,00	2.400,00	100 %

3.1 Cómo leer estas cifras

- **Peso neto de datos:** 1.614 MB (1,58 GB) para 10.565.088 filas, es decir, cerca de 160 bytes por fila en promedio.
- **Índices:** 786 MB, equivalente al 32,75 % del espacio total (2.400 MB). En las tres tablas grandes el índice pesa alrededor del 49 % del dato (game_events 48,5 %, appearances 49,8 %, player_valuations 49,4 %).
- **Concentración:** game_events por sí sola es el 68 % de las filas y el 75,5 % del espacio total, por eso es la tabla candidata a particionamiento y a ser el origen de Fact_Events.
- **Justificación del costo de índices:** el espacio extra se paga con menores tiempos en JOINS, filtros multidimensionales y agregaciones, que son el patrón de uso de un DWH.
- **En Power BI:** con el motor columnar VertiPaq (codificación por diccionario y compresión RLE) los 2.400 MB se reducen a unos 180 a 220 MB, una reducción aproximada de 11 veces, lo que permite el modo Import y respuestas rápidas.

Verificación aritmética. La suma de registros por tabla da 10.565.088; la suma de datos da 1.614 MB; la de índices, 786 MB; y $786 / 2.400 = 32,75 \%$. Las cifras de la propuesta son consistentes entre sí.

3.2 Consultas para reproducir la medición en vivo

```
-- Tamano de datos e indices por tabla
SELECT c.relname AS tabla,
       c.reltuples::bigint AS filas_estimadas,
       pg_size_pretty(pg_relation_size(c.oid)) AS datos,
       pg_size_pretty(pg_indexes_size(c.oid)) AS indices,
       pg_size_pretty(pg_total_relation_size(c.oid)) AS total
FROM pg_class c JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE n.nspname = 'silver' AND c.relkind = 'r'
ORDER BY pg_total_relation_size(c.oid) DESC;

-- Conteo exacto (para la diapositiva de volumetria)
SELECT 'game_events' AS tabla, COUNT(*) FROM silver.game_events
UNION ALL SELECT 'appearances', COUNT(*) FROM silver.appearances;
```

CAPITULO 4

La base de datos de origen

Las 7 entidades de Transfermarkt, su significado y sus relaciones

Los datos provienen de archivos CSV del ecosistema Transfermarkt. Se organizan en **tres entidades maestras** (competitions, clubs, players), **una de contexto** (games) y **tres de hechos transaccionales** (appearances, game_events, player_valuations). Las columnas listadas son las principales; antes de la sustentación conviene contrastarlas con el encabezado real de cada CSV.

Tabla	Qué representa	Llave	Columnas principales
competitions	Ligas y copas	competition_id	name, type, country_name, confederation
clubs	Equipos	club_id	name, domestic_competition_id, stadium_name, squad_size, average_age, total_market_value
players	Futbolistas	player_id	name, position, sub_position, date_of_birth, country_of_citizenship, foot, height_in_cm, current_club_id, market_value_in_eur
games	Partidos jugados	game_id	competition_id, season, round, date, home_club_id, away_club_id, home_club_goals, away_club_goals, referee, stadium, attendance
appearances	Participación de un jugador en un partido	appearance_id	game_id, player_id, player_club_id, date, goals, assists, yellow_cards, red_cards, minutes_played
game_events	Eventos dentro del partido	game_event_id	game_id, minute, type, club_id, player_id, description, player_in_id, player_assist_id
player_valuations	Valor de mercado de un jugador en una fecha	player_id + date	date, market_value_in_eur, current_club_id

4.1 Relaciones entre tablas (cardinalidad)

- competitions 1 : N games (un partido pertenece a una competición).
- clubs 1 : N games dos veces: como local (home_club_id) y como visitante (away_club_id).
- clubs 1 : N players (club actual del jugador).
- games 1 : N appearances y games 1 : N game_events.
- players 1 : N appearances, players 1 : N game_events y players 1 : N player_valuations.

Idea clave para explicar. El esquema de origen es relacional y normalizado: cada pregunta analítica requiere de 4 a 6 JOINS. El Data Warehouse pre-organiza esos datos alrededor de un hecho (el evento) y de dimensiones planas, de modo que la mayoría de consultas necesite un solo nivel de JOIN.

4.2 Problema OLTP frente a solución OLAP

Aspecto	OLTP (origen)	OLAP (Data Warehouse)
Objetivo	Registrar y consultar filas individuales	Analizar grandes volúmenes históricos
Modelo	Normalizado (3FN)	Dimensional en estrella
Consulta típica	Muchos JOINS, pocos registros	Agregaciones sobre millones de filas
Optimiza	Escritura y consistencia	Lectura y agregación

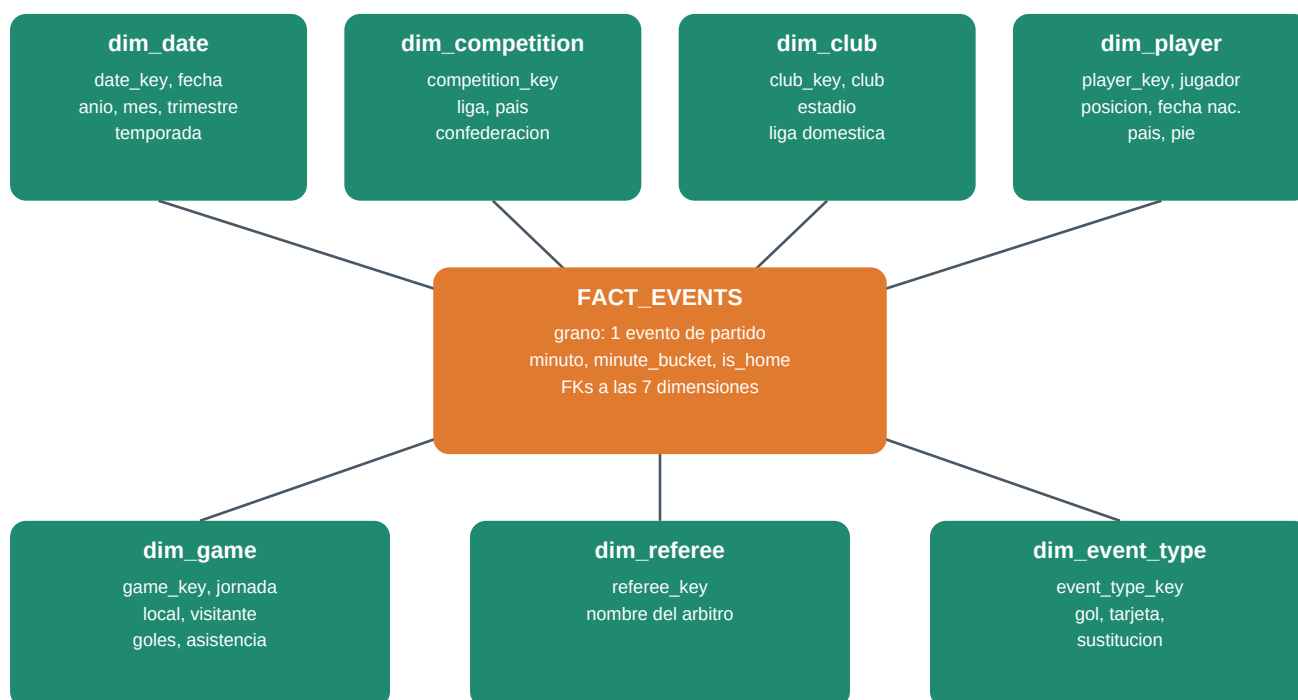
Aspecto	OLTP (origen)	OLAP (Data Warehouse)
Historia	Estado actual	Histórico completo por temporada

CAPITULO 5

Diseño del modelo estrella

Sustentación del 30 de septiembre

El modelo tiene un **hecho principal**, Fact_Events, rodeado de **siete dimensiones**. Se complementa con dos hechos secundarios (Fact_Apearances y Fact_Valuations) que comparten las mismas dimensiones (dimensiones conformadas), lo que permite cruzar eventos, rendimiento y valor de mercado en un solo tablero.



Nota de diseño: el grano de la tabla de hechos principal es un evento de partido (gol, tarjeta o sustitución). Las dimensiones descritas son la propuesta del grupo y deben validarse contra el modelo final implementado.

5.1 Tablas de hechos

Hecho	Grano	Medidas	Responde a
Fact_Events	Un evento de un partido	conteo de eventos, minuto, minute_bucket	RF03, RF04, RF06, RF07
Fact_Apearances	Un jugador en un partido	goles, asistencias, amarillas, rojas, minutos	RF01, RF02, RF06, RF08
Fact_Valuations	Un jugador en una fecha	valor de mercado en euros	RF05

5.2 Dimensiones, atributos y jerarquías

Dimensión	Atributos principales	Jerarquía
dim_date	fecha, año, trimestre, mes, semana, temporada	Año > Mes > Partido > Minuto (el minuto vive en el hecho)
dim_competition	nombre, tipo, país, confederación	Confederación > Liga

Dimensión	Atributos principales	Jerarquía
dim_club	nombre, estadio, liga doméstica, tamaño de plantilla	Liga > Club
dim_player	nombre, posición, subposición, fecha de nacimiento, país, pie, estatura	Club > Jugador; posición para RF02
dim_game	temporada, jornada, local, visitante, goles, estadio, asistencia	Temporada > Jornada > Partido
dim_referee	nombre del árbitro	Árbitro
dim_event_type	tipo de evento (gol, tarjeta, sustitución) y subtipo	Tipo > Subtipo

5.3 Decisiones de diseño que debes saber defender

- **Llaves sustitutas (surrogate keys):** las dimensiones usan un entero propio (xxx_key) y conservan el id de negocio (xxx_id) con restricción UNIQUE. Así el DWH no depende de cambios en el origen y los JOINS son más rápidos.
- **Dimensiones conformadas:** dim_player, dim_club, dim_date y dim_competition se comparten entre los tres hechos.
- **Cambios lentos (SCD):** se propone Tipo 1 (sobrescribir) por simplicidad. Si se quiere historial de cambios de club, dim_player pasa a Tipo 2 con fecha de vigencia.
- **minute_bucket:** intervalo de 15 minutos calculado en el ETL (0 a 14, 15 a 29, ..., 90 o más), requisito RF07.
- **Partición:** Fact_Events se particiona por rango de fecha para reducir I/O (RNF08).

5.4 Scripts DDL de la capa Oro (núcleo)

```

CREATE SCHEMA IF NOT EXISTS gold;

CREATE TABLE gold.dim_date (
    date_key INT PRIMARY KEY,          -- AAAAMMDD
    full_date DATE NOT NULL, year SMALLINT, quarter SMALLINT,
    month SMALLINT, month_name VARCHAR(12), week SMALLINT, season VARCHAR(7));

CREATE TABLE gold.dim_competition (
    competition_key SERIAL PRIMARY KEY, competition_id VARCHAR(10) UNIQUE NOT NULL,
    name VARCHAR(100), type VARCHAR(30), country_name VARCHAR(60), confederation VARCHAR(20));

CREATE TABLE gold.dim_club (
    club_key SERIAL PRIMARY KEY, club_id INT UNIQUE NOT NULL, name VARCHAR(100),
    competition_key INT REFERENCES gold.dim_competition, stadium_name VARCHAR(100), squad_size SMALLINT);

CREATE TABLE gold.dim_player (
    player_key SERIAL PRIMARY KEY, player_id INT UNIQUE NOT NULL, name VARCHAR(120),
    position VARCHAR(30), sub_position VARCHAR(40), date_of_birth DATE,
    country_of_citizenship VARCHAR(60), foot VARCHAR(10), height_in_cm SMALLINT);

CREATE TABLE gold.dim_referee (referee_key SERIAL PRIMARY KEY, referee_name VARCHAR(100) UNIQUE);
CREATE TABLE gold.dim_event_type (event_type_key SERIAL PRIMARY KEY, event_type VARCHAR(30), event_subtype VARCHAR(60));

CREATE TABLE gold.dim_game (
    game_key SERIAL PRIMARY KEY, game_id INT UNIQUE NOT NULL,
    competition_key INT REFERENCES gold.dim_competition, season VARCHAR(7), round VARCHAR(30),
    game_date DATE, home_club_key INT REFERENCES gold.dim_club, away_club_key INT REFERENCES gold.dim_club,
    home_goals SMALLINT, away_goals SMALLINT, attendance INT, referee_key INT REFERENCES gold.dim_referee);

CREATE TABLE gold.fact_events (
    event_key BIGSERIAL, date_key INT NOT NULL REFERENCES gold.dim_date,
    game_key INT NOT NULL REFERENCES gold.dim_game, competition_key INT REFERENCES gold.dim_competition,
    club_key INT REFERENCES gold.dim_club, player_key INT REFERENCES gold.dim_player,
    assist_player_key INT REFERENCES gold.dim_player, referee_key INT REFERENCES gold.dim_referee,
    event_type_key INT NOT NULL REFERENCES gold.dim_event_type,
    minute SMALLINT, minute_bucket SMALLINT, is_home BOOLEAN,
    PRIMARY KEY (event_key, date_key)
) PARTITION BY RANGE (date_key);

```

CAPITULO 6

Arquitectura Medallón

Bronce, Plata y Oro: roles, actividades y herramientas

La Arquitectura Medallón organiza el dato en capas de calidad creciente. Cada capa es un esquema de PostgreSQL (bronze, silver, gold), de modo que el flujo es trazable y cada transformación es reproducible.



Capa	Contenido	Reglas principales	Quién la usa
Bronce (bronze)	Copia fiel de los CSV: todas las columnas como TEXT, más _load_id, _loaded_at y _source_file	No se corrige nada; solo se registra. Tolera errores de codificación y delimitadores	Ingeniero de datos
Plata (silver)	Tablas tipadas, sin duplicados, con PK y FK	Fechas ISO-8601, tipado fuerte, nulos canónicos, cero huérfanos	Ingeniero de datos y calidad
Oro (gold)	Modelo estrella y tablas agregadas	Llaves sustitutas, jerarquías, índices, particiones, vistas materializadas	Analistas y Power BI

6.1 Roles que colaboran en cada etapa

Etapas	Rol	Responsabilidad
Bronce	Ingeniero de datos	Ingester archivos, controlar errores de lectura, registrar auditoría de carga
Plata	Ingeniero de datos y responsable de calidad de datos	Limpiar, tipar, imputar nulos, validar integridad referencial, documentar reglas EDA
Oro	Modelador de datos y administrador de base de datos	Diseñar la estrella, crear índices, particiones y agregaciones
Consumo	Analista de BI y usuarios de negocio	Construir el dashboard y obtener conclusiones para decisiones

6.2 Actividades del ingeniero en cada capa

Capa	Actividades
Bronce	1) Leer cada CSV por chunks. 2) Intentar utf-8 y, si falla, latin-1. 3) Cargar con COPY a tablas TEXT. 4) Registrar filas leídas y errores en meta.etl_audit.
Plata	1) Convertir tipos y fechas. 2) Aplicar reglas de nulos. 3) Eliminar duplicados por llave de negocio. 4) Descartar o aislar filas inválidas. 5) Insertar con ON CONFLICT para idempotencia. 6) Verificar FKs.
Oro	1) Poblar dimensiones y asignar llaves sustitutas. 2) Cargar hechos resolviendo FKs. 3) Calcular minute_bucket y is_home. 4) Crear índices, particiones y vistas materializadas. 5) Refrescar agregaciones.

6.3 Herramientas implementadas

Herramienta	Uso en el proyecto
PostgreSQL	Motor del DWH, esquemas bronze/silver/gold, índices B-Tree/Hash, particionamiento, vistas materializadas
Python (pandas, psycopg2)	Pipeline ETL por chunks, validaciones y carga masiva con COPY
SQL	Transformaciones Plata a Oro, agregaciones y pruebas de calidad
Power BI Desktop	Modelo semántico, medidas DAX y dashboard interactivo
pg_class / pg_total_relation_size	Auditoría física de almacenamiento
Si el grupo utiliza otras librerías o herramientas (por ejemplo SQLAlchemy, un orquestador o un cliente como DBeaver o pgAdmin), agrégalas en esta tabla y en la diapositiva 10.	

Reglas del proceso EDA

Análisis exploratorio y reglas de calidad aplicadas antes de Plata

El EDA (Exploratory Data Analysis) detecta problemas en el dato crudo y define reglas que el ETL aplica. Cada regla tiene un identificador para citarla en la sustentación y se registra en el linaje.

ID	Regla	Acción del ETL	Verificación SQL (ejemplo)
E01	Nulos en métricas numéricas (goles, minutos, tarjetas)	Imputar 0	SELECT COUNT(*) FROM silver.appearances WHERE goals IS NULL;
E02	Nulos en valor de mercado	Mantener NULL (no se inventa valor) y marcar en auditoría	SELECT COUNT(*) FROM silver.player_valuations WHERE market_value_in_eur IS NULL;
E03	Nulos en textos (árbitro, país)	Sustituir por 'Desconocido'	SELECT COUNT(*) FROM silver.games WHERE referee IS NULL;
E04	Fechas en formatos mixtos	Convertir a ISO-8601 (YYYY-MM-DD); descartar si no se interpreta	SELECT COUNT(*) FROM silver.games WHERE date IS NULL;
E05	Duplicados por llave de negocio	Conservar un registro por llave (ON CONFLICT)	SELECT game_id, COUNT(*) FROM silver.games GROUP BY 1 HAVING COUNT(*)>1;
E06	Rangos inválidos (minutos fuera de 0 a 130, edades negativas, valor menor a 0)	Aislar en tabla de rechazados y contar como descartados	SELECT COUNT(*) FROM silver.game_events WHERE minute NOT BETWEEN 0 AND 130;
E07	Registros huérfanos (FK sin padre)	Descartar o aislar; objetivo final 0	SELECT COUNT(*) FROM silver.appearances a LEFT JOIN silver.players p USING(player_id) WHERE p.player_id IS NULL;
E08	Codificación o delimitador corrupto	Reintentar con latin-1 y omitir líneas malformadas registrándolas	Contadores en meta.etl_audit
E09	Tipos incorrectos (texto en columna numérica)	Convertir con error controlado a NULL y contar	Comparar filas leídas contra insertadas
E10	Consistencia de marcador (suma de eventos de gol frente a goles del partido)	Reportar diferencias para revisión	Consulta comparativa game_events frente a games

Las reglas anteriores son la propuesta de referencia; ajusta los umbrales (por ejemplo, el rango de minutos) a lo que el grupo realmente observó en el EDA.

CAPITULO 8

Proceso ETL

Sustentación del 12 de octubre: herramientas y flujo

El pipeline Python extrae los CSV, transforma por capas y carga en PostgreSQL. Está diseñado para ser **idempotente** (reejecutarlo no duplica datos), **tolerante a fallos** y **auditable**.

8.1 Flujo paso a paso

Paso	Fase	Qué ocurre
1	Extracción	Lectura de cada CSV con chunksize parametrizado (10.000 a 200.000 filas), dtype=str, codificación utf-8 con respaldo latin-1
2	Carga a Bronce	COPY desde buffer en memoria; TRUNCATE previo de la tabla bronze para idempotencia
3	Transformación a Plata	Tipado, ISO-8601, nulos canónicos, eliminación de duplicados, validación de rangos y FKs; INSERT ... ON CONFLICT DO UPDATE
4	Carga a Oro	Poblar dimensiones, resolver llaves sustitutas, cargar hechos, calcular minute_bucket e is_home
5	Agregaciones	REFRESH MATERIALIZED VIEW de las tablas gold.agg_*
6	Auditoría	Registrar por fase filas leídas, transformadas, descartadas e insertadas en meta.etl_audit

8.2 Tabla de auditoría y linaje (RF11)

```
CREATE SCHEMA IF NOT EXISTS meta;
CREATE TABLE meta.etl_audit (
  audit_id BIGSERIAL PRIMARY KEY, run_id UUID NOT NULL, layer VARCHAR(10) NOT NULL,
  table_name VARCHAR(60) NOT NULL, rows_read BIGINT, rows_transformed BIGINT,
  rows_discarded BIGINT, rows_inserted BIGINT, status VARCHAR(10),
  started_at TIMESTAMP DEFAULT now(), finished_at TIMESTAMP);
```

8.3 Esqueleto del pipeline en Python

```

import io, uuid, pandas as pd, psycpg2

CHUNK = 100_000
RUN_ID = str(uuid.uuid4())

def read_chunks(path):
    for enc in ("utf-8", "latin-1"):          # RNF09: tolerancia a codificacion
        try:
            for ch in pd.read_csv(path, dtype=str, chunksize=CHUNK, encoding=enc,
                                   on_bad_lines="skip", keep_default_na=False):
                yield ch
            return
        except UnicodeDecodeError:
            continue

def load_bronze(conn, path, table):
    cur = conn.cursor(); cur.execute(f"TRUNCATE bronze.{table}") # RNF07: idempotencia
    read = 0
    for ch in read_chunks(path):
        buf = io.StringIO(); ch.to_csv(buf, index=False, header=False)
        buf.seek(0)
        cur.copy_expert(f"COPY bronze.{table} FROM STDIN WITH CSV", buf)
        read += len(ch)
    conn.commit(); return read

def to_silver_appearances(conn):
    cur = conn.cursor()
    cur.execute('''
        INSERT INTO silver.appearances (appearance_id, game_id, player_id, date, goals, assists,
                                         yellow_cards, red_cards, minutes_played)
        SELECT appearance_id, game_id::int, player_id::int, NULLIF(date, '')::date,
               COALESCE(NULLIF(goals, '')::int, 0), COALESCE(NULLIF(assists, '')::int, 0),
               COALESCE(NULLIF(yellow_cards, '')::int, 0), COALESCE(NULLIF(red_cards, '')::int, 0),
               COALESCE(NULLIF(minutes_played, '')::int, 0)
        FROM bronze.appearances b
        WHERE EXISTS (SELECT 1 FROM silver.players p WHERE p.player_id = b.player_id::int)
        ON CONFLICT (appearance_id) DO UPDATE SET goals = EXCLUDED.goals,
            assists = EXCLUDED.assists, minutes_played = EXCLUDED.minutes_played''')
    conn.commit()

```

Este código es un esqueleto para explicar la lógica. Adáptalo a los nombres reales de columnas y a la estructura de carpetas del proyecto antes de mostrarlo.

CAPITULO 9

Capa Oro: agregaciones, índices y particiones

RF10, RNF04 y RNF08

Las agregaciones precalculadas evitan recorrer 7 millones de eventos en cada clic del dashboard. Se materializan como vistas y se refrescan al final del ETL.

```
-- RF01 / RF08: rendimiento por jugador y temporada (incluye edad para sub-23)
CREATE MATERIALIZED VIEW gold.agg_player_season AS
SELECT fa.player_key, dd.season, fa.competition_key,
       SUM(fa.goals) goals, SUM(fa.assists) assists, SUM(fa.minutes_played) minutes,
       SUM(fa.yellow_cards) yellow, SUM(fa.red_cards) red,
       MIN(EXTRACT(YEAR FROM age(dd.full_date, dp.date_of_birth))):int AS age
FROM gold.fact_appearances fa
JOIN gold.dim_date dd ON dd.date_key = fa.date_key
JOIN gold.dim_player dp ON dp.player_key = fa.player_key
GROUP BY fa.player_key, dd.season, fa.competition_key;

-- RF06: tarjetas por 90 minutos
SELECT dp.name, (SUM(yellow)+SUM(red))*90.0/NULLIF(SUM(minutes),0) AS cards_per_90
FROM gold.agg_player_season a JOIN gold.dim_player dp USING (player_key)
GROUP BY dp.name HAVING SUM(minutes) > 900 ORDER BY 2 DESC;

-- RF07: eventos por intervalo de 15 minutos
CREATE MATERIALIZED VIEW gold.agg_events_15min AS
SELECT competition_key, event_type_key, minute_bucket, COUNT(*) n_events
FROM gold.fact_events GROUP BY 1,2,3;

-- RF05: correlacion valor de plantilla vs puntos (por temporada)
SELECT season, corr(squad_value, points) AS correlacion
FROM gold.agg_club_season GROUP BY season;

-- RNF08: indices y particiones
CREATE TABLE gold.fact_events_2023 PARTITION OF gold.fact_events
FOR VALUES FROM (20230701) TO (20240701);
CREATE INDEX ix_fe_game ON gold.fact_events (game_key);
CREATE INDEX ix_fe_player ON gold.fact_events (player_key);
CREATE INDEX ix_fe_type ON gold.fact_events USING HASH (event_type_key);
```

9.1 Cómo se calculan los puntos (RF05)

Cada partido genera dos filas de puntos (una por club): 3 puntos para el ganador, 1 para cada club en empate y 0 para el perdedor, a partir de home_goals y away_goals de dim_game. Se suman por club y temporada en gold.agg_club_season junto con el valor total de la plantilla tomado de Fact_Valuations. La correlación de Pearson se calcula con corr() de PostgreSQL.

CAPITULO 10

Dashboard Power BI y storytelling

Sustentación del 14 de octubre

El dashboard se conecta a la capa Oro con el conector nativo de PostgreSQL en modo **Import** (VertiPaq), lo que garantiza respuestas menores a 3 segundos. Se diseña para **responder preguntas de decisión**, no solo para mostrar gráficos.

10.1 Páginas del tablero

Página	Pregunta de negocio	Visuales	RF
1. Resumen ejecutivo	¿Cómo está el ecosistema esta temporada?	Tarjetas KPI (goles, partidos, tarjetas por 90, valor total), tendencia por temporada	RF03, RF09
2. Rankings y scouting	¿Quiénes son los mejores jugadores y los talentos sub-23?	Top N con parámetro, segmentador de edad, dispersión valor frente a rendimiento	RF01, RF08
3. Local vs visitante	¿Cuánto pesa jugar en casa?	Barras comparativas, matriz por liga	RF04
4. Disciplina y árbitros	¿Dónde y con quién hay más tarjetas?	Ranking de árbitros, tarjetas por 90, mapa de calor por liga	RF06
5. Valor y puntos	¿El dinero compra resultados?	Dispersión valor de plantilla frente a puntos y coeficiente de correlación	RF05
6. Momentos del partido	¿En qué minutos ocurren los eventos?	Columnas por intervalo de 15 minutos	RF07

10.2 Filtros claros y navegación

- Segmentadores globales sincronizados en todas las páginas: Temporada, Confederación/Liga, Club, Posición y Rango de edad.
- Jerarquías con drill-down: Confederación > Liga > Club > Jugador y Año > Mes > Partido.
- Botón 'Restablecer filtros' y navegación por pestañas; máximo 6 visuales por página.
- Colores consistentes: un color para local y otro para visitante en todo el reporte.

10.3 Medidas DAX de referencia

```
Goles = SUM(fact_appearances[goals])
Tarjetas por 90 = DIVIDE(SUM(fact_appearances[yellow_cards]) + SUM(fact_appearances[red_cards]),
    SUM(fact_appearances[minutes_played])) * 90
Valor de plantilla = SUM(fact_valuations[market_value_eur])
Es sub-23 = IF(MAX(agg_player_season[age]) < 23, "Sub-23", "23 o mas")
```

10.4 Guion de storytelling

Acto	Mensaje	Ejemplo de frase
Contexto	Qué datos tenemos y su escala	'Analizamos más de 10,5 millones de registros de fútbol profesional.'
Hallazgo	Qué patrón aparece	'Los eventos se concentran en ciertos intervalos y el local tiene ventaja.'
Explicación	Por qué ocurre	'El valor de la plantilla explica parte de los puntos, pero no todo.'

Acto	Mensaje	Ejemplo de frase
Decisión	Qué haría un club o scout	'Priorizar jóvenes sub-23 con alto rendimiento por minuto y bajo valor.'

CAPITULO 11

Guía por diapositiva

Qué dice cada diapositiva y cómo explicarla

La presentación tiene 13 diapositivas. Las diapositivas 1 a 5 corresponden a la presentación de 5 minutos del 28 de septiembre (aprox. 1 minuto cada una); la 6, a la sustentación del 30 de septiembre; y la 7 a la 13, a la presentación del 7 de octubre, que incluye también el proceso ETL (diapositivas 12 y 13).

Diapositiva 1. Introducción

28 sep

Qué muestra. Presenta el problema: el fútbol decide con datos, pero el esquema OLTP es lento para analizar millones de filas. Anuncia la solución: DWH en PostgreSQL con Arquitectura Medallón y Power BI.

Cómo explicarla. Empieza con una frase gancho: 'Hoy los clubes fichan con datos'. Explica que Transfermarkt tiene transferencias, perfiles y eventos; que un OLTP se atasca con consultas multidimensionales; y que por eso se construye un sistema OLAP. Cierra mencionando las tres capas.

Pregunta probable. ¿Por qué no usar la base transaccional directamente? Porque está optimizada para escribir y consultar filas, no para agregar millones de registros con múltiples JOINS.

Diapositiva 2. Objetivo general

28 sep

Qué muestra. Un solo objetivo: diseñar e implementar la arquitectura Medallón con modelo estrella sobre más de 10,5 millones de registros para analítica en Power BI.

Cómo explicarla. Léelo con tus palabras y resáltalo en tres ideas: arquitectura escalable, modelo estrella, volumen de más de 10,5 millones de registros. Termina con 'todo para tomar decisiones en Power BI'.

Pregunta probable. ¿Qué significa escalable? Que al crecer los datos el rendimiento no se degrada gracias a índices, particiones y agregaciones.

Diapositiva 3. Objetivos específicos

28 sep

Qué muestra. Cuatro objetivos que forman el flujo del proyecto: ingesta, modelo dimensional, ETL y visualización.

Cómo explicarla. Recorre las cuatro tarjetas como una cadena: primero entran los datos, luego se modelan, luego el ETL los mueve y finalmente se visualizan. Menciona el criterio medible de cada uno (cero huérfanos, Fact_Events, chunks, drill-down).

Pregunta probable. ¿Cómo miden que cumplieron? Con evidencias: conteos, consulta de huérfanos, scripts DDL, logs del ETL y demostración del dashboard.

Diapositiva 4. Requerimientos

28 sep

Qué muestra. Resume los 11 requerimientos funcionales (qué analiza el sistema) y los 10 no funcionales (qué calidad debe tener).

Cómo explicarla. No leas todos. Agrupa: los funcionales son preguntas de negocio (rankings, disciplina, local vs visitante, scouting, valor frente a puntos); los no funcionales son garantías técnicas (chunks, integridad referencial, idempotencia, menos de 3 s).

Pregunta probable. ¿Qué es idempotencia? Que ejecutar el ETL varias veces deja la base en el mismo estado, sin duplicados.

Diapositiva 5. Volumetría	28 sep
Qué muestra. Muestra la medición física real: 10.565.088 filas, 1.614 MB de datos, 786 MB de índices, 2.400 MB en total.	
Cómo explicarla. Destaca que game_events concentra el 68 % de las filas. Explica que el índice cuesta 32,75 % del espacio pero acelera JOINS y filtros. Menciona que en Power BI VertiPaq comprime todo a 180 a 220 MB.	
Pregunta probable. ¿De dónde salen esas cifras? De pg_class, pg_relation_size() y pg_total_relation_size() en PostgreSQL.	
Diapositiva 6. Modelo estrella	30 sep
Qué muestra. Fact_Events en el centro con las dimensiones fecha, competición, club, jugador, partido, árbitro y tipo de evento; más dos hechos complementarios.	
Cómo explicarla. Comienza por el grano: un evento de partido. Luego recorre las dimensiones y sus jerarquías. Aclara que las dimensiones son conformadas y que se usan llaves sustitutas. Cierra relacionando el modelo con los requerimientos (RF03, RF04, RF06, RF07).	
Pregunta probable. ¿Por qué estrella y no copo de nieve? Porque reduce JOINS y acelera consultas analíticas a costa de algo de redundancia aceptable.	
Diapositiva 7. Arquitectura Medallón	7 oct
Qué muestra. Tres capas: Bronce (crudo), Plata (validado y tipado), Oro (modelo de negocio y agregaciones).	
Cómo explicarla. Explica el viaje de un dato: CSV a Bronce sin cambios, a Plata limpio y con FKs, a Oro en estrella. Subraya que cada capa es un esquema de PostgreSQL y que permite reprocesar sin perder el original.	
Pregunta probable. ¿Por qué tres capas? Para aislar errores, conservar el dato original y entregar al negocio solo dato confiable.	
Diapositiva 8. Roles por etapa	7 oct
Qué muestra. Quién colabora en cada capa: ingeniero de datos, calidad, modelador/DBA y analista de BI.	
Cómo explicarla. Relaciona cada rol con su capa y su entregable. Si el grupo se repartió tareas, indica quién asumió cada rol.	
Pregunta probable. ¿Quién valida la calidad? El responsable de calidad con las reglas EDA y las pruebas de integridad.	
Diapositiva 9. Actividades por capa	7 oct
Qué muestra. Lo que hace el ingeniero en cada capa: ingesta, depuración y modelado/optimización.	
Cómo explicarla. Usa verbos: en Bronce cargo y registro; en Plata limpio, tipo y valido; en Oro modelo, indexo y agrego.	
Pregunta probable. ¿Qué hace en Bronce si un CSV viene corrupto? Reintenta con otra codificación, omite líneas malformadas y las cuenta en la auditoría.	
Diapositiva 10. Herramientas	7 oct
Qué muestra. PostgreSQL, Python, Power BI y el modelo dimensional como herramientas del proyecto.	
Cómo explicarla. Explica el rol de cada una en una frase y por qué se eligió (PostgreSQL por robustez e indexación, Python por flexibilidad, Power BI por interactividad).	
Pregunta probable. ¿Por qué Python y no solo SQL? Porque maneja lectura por chunks, errores de archivo y orquestación del flujo.	

Diapositiva 11. Reglas EDA	7 oct
Qué muestra. Seis familias de reglas: nulos, tipos y fechas, duplicados, integridad, rangos válidos y linaje.	
Cómo explicarla. Muestra que el EDA no es teoría: cada regla tiene una acción concreta y una consulta de verificación (capítulo 7). Cita un ejemplo: nulos en goles se imputan en 0 y nulos en valor de mercado se conservan.	
Pregunta probable. ¿Por qué no imputar el valor de mercado? Porque inventar un valor distorsionaría la correlación con los puntos.	
Diapositiva 12. Proceso ETL: flujo	7 oct
Qué muestra. Las cuatro fases del flujo: extraer los CSV por chunks, cargar a Bronce, transformar a Plata y cargar a Oro, con auditoría en meta.etl_audit.	
Cómo explicarla. Cuenta el viaje de un archivo, por ejemplo appearances.csv: se lee en bloques de 100.000 filas, se copia crudo a Bronce, se tipa y depura en Plata y se convierte en hechos y dimensiones en Oro. Cierra con la auditoría: filas leídas, transformadas, descartadas e insertadas (capítulo 8).	
Pregunta probable. ¿Qué pasa si el ETL falla a la mitad? Al ser idempotente se reejecuta completo: Bronce se vacía y Plata y Oro usan UPSERT, sin duplicar.	
Diapositiva 13. ETL: herramientas y garantías	7 oct
Qué muestra. Herramientas (Python, SQL, COPY) y tres garantías: procesamiento por chunks, idempotencia y tolerancia a fallos.	
Cómo explicarla. Relaciona cada garantía con su requerimiento: chunks con RNF01, idempotencia con RNF07 y tolerancia a fallos con RNF09. Si pueden, muestren dos corridas seguidas con el mismo conteo de filas.	
Pregunta probable. ¿Por qué COPY y no INSERT fila a fila? COPY es mucho más rápido para cargas masivas de millones de registros.	

Banco de preguntas para la sustentación

Respuestas cortas y precisas

1. ¿Qué es un Data Warehouse y en qué se diferencia de una base de datos operacional?

Es un repositorio histórico e integrado, modelado para análisis (OLAP). Una base operacional (OLTP) está normalizada y optimizada para escrituras y consultas de pocas filas.

2. ¿Qué es la Arquitectura Medallón?

Un patrón en capas: Bronce guarda el dato crudo, Plata lo limpia y valida, Oro lo modela para negocio. Aporta trazabilidad, reprocesamiento y calidad progresiva.

3. ¿Por qué un modelo estrella?

Porque reduce la cantidad de JOINS, es fácil de entender para analistas y es el modelo que mejor aprovecha Power BI.

4. ¿Cuál es el grano de Fact_Events y por qué importa?

Un evento de un partido. Definir el grano evita mezclar niveles de detalle y garantiza agregaciones correctas.

5. ¿Qué son las llaves sustitutas?

Enteros propios del DWH que reemplazan al id de negocio como PK de las dimensiones: independizan el DWH del origen y aceleran los JOINS.

6. ¿Cómo garantizan que no hay huérfanos?

Con FOREIGN KEY en Plata y Oro y una prueba de LEFT JOIN que debe devolver 0 filas.

7. ¿Cómo logran que el ETL sea idempotente?

TRUNCATE en Bronce y UPSERT (INSERT ... ON CONFLICT DO UPDATE) en Plata y Oro; así reejecutar no duplica.

8. ¿Por qué procesar por chunks?

Para no cargar millones de filas en memoria RAM; cada bloque de 10.000 a 200.000 filas se procesa y se carga con COPY.

9. ¿Cómo cumplen que las consultas tarden menos de 3 segundos?

Power BI en modo Import (VertiPaq comprime a 180 a 220 MB), tablas agregadas en Oro e índices B-Tree/Hash.

10. ¿Por qué el 32,75 % del espacio son índices y eso es aceptable?

Porque es el costo de acelerar JOINS y filtros; en un DWH se lee mucho más de lo que se escribe.

11. ¿Cómo calculan puntos y la correlación valor-puntos?

3, 1 y 0 puntos por victoria, empate y derrota desde los goles de cada partido; luego corr() de PostgreSQL entre valor de plantilla y puntos por temporada.

12. ¿Cómo se calcula tarjetas por 90 minutos?

(amarillas + rojas) x 90 / minutos jugados, con un mínimo de minutos para evitar ruido.

13. ¿Cómo trataron los nulos?

Con reglas canónicas: métricas numéricas a 0, textos a 'Desconocido' y el valor de mercado se conserva como NULL para no distorsionar el análisis.

14. ¿Cómo llegaron a más de 10,5 millones de registros?

Respuesta a preparar con el grupo: describan el proceso de 'expansión controlada' del dataset y cómo se mantuvo la integridad referencial.

Puntos de atención y lista de verificación

Lo que debes revisar antes de sustentar

13.1 Alertas sobre el alcance

RF02 y RF04. Piden pases precisos, faltas, tiros a puerta, recuperaciones y posesión. En la versión pública del dataset de Transfermarkt las tablas de apariciones y eventos solo traen goles, asistencias, tarjetas y minutos; esas métricas técnicas no existen. Verifiquen si su dataset las incluye. Si no, ajusten el alcance (por ejemplo, métricas por posición con goles, asistencias, tarjetas y minutos) y expliquen la decisión con transparencia.

Expansión a más de 10,5 millones. El objetivo 1 habla de 'expansión controlada'. Preparen una explicación clara de cómo se generó el volumen adicional y cómo se comprobó que no hay huérfanos, porque es una pregunta casi segura.

Coherencia entre documento y base real. Los nombres de columnas, dimensiones y reglas de este documento son una propuesta de referencia. Contrástenlos con los scripts reales del grupo y ajusten lo que difiera.

13.2 Lista de verificación por entrega

Entrega	Verificar
28 sep	5 diapositivas en 5 minutos; cifras de volumetría correctas; un expositor por bloque
30 sep	Diagrama de la estrella visible; grano, dimensiones y jerarquías explicados; DDL listo para mostrar
7 oct	Tres capas implementadas y consultables; tabla de roles; herramientas y reglas EDA con ejemplos
12 oct	Pipeline ejecutado en vivo o con log; demostración de idempotencia (dos corridas, mismo conteo); tabla de auditoría
14 oct	Dashboard con 6 páginas, segmentadores sincronizados, drill-down y tiempos menores a 3 s; guion de storytelling
Final	Este documento actualizado con capturas, scripts finales y resultados reales

13.3 Glosario

Término	Significado
OLTP / OLAP	Procesamiento transaccional / analítico en línea
ETL	Extracción, Transformación y Carga
Hecho	Tabla con medidas numéricas de un proceso (eventos, apariciones)
Dimensión	Tabla descriptiva para filtrar y agrupar (jugador, club, fecha)
Grano	Nivel de detalle de una fila de la tabla de hechos
Llave sustituta	Identificador entero propio del DWH
Idempotencia	Reejecutar produce el mismo resultado
VertiPaq	Motor columnar en memoria de Power BI
Linaje	Registro del recorrido y conteos del dato entre capas